B.Tech Project (CP302)

Title: Perpetual voting method using pairwise comparison

Group: 15

Supervised By:
Dr. Manish Kumar

Submitted By:
Radha (2022CSB1108)
Shaikh Asra Swaleh (2022CSB1121)

Contents

INTRODUCTION

- **Voting systems:** Voting systems are formal methods for aggregating individual preferences into a collective decision.
- **Plurality**(winner is the most first-place votes) and **Borda Count**(points based on rankings) are simple, but can give unfair results such as candidate preferred by most voters overall may lose.
- **Condorcet Method:** A candidate who beats every other candidate in pairwise comparison is called the Condorcet winner. The method fails when no such candidate exists (**cycle:** A > B, B > C, C > A → **Condorcet paradox**).
- **Schulze Method:** A Condorcet-compliant algorithm that resolves cycles by computing the **strongest paths** between candidates and selecting the winner consistently.
- **Perpetual Method:** An extension for repeated elections. It ensures fairness across multiple cycles by carrying over voter representation, preventing dominance by a single group, and maintaining long-term proportionality.

Problem Statement & Objectives

- **Traditional voting systems** often face serious issues:
  - **Cyclic preferences(Condorcet paradox):** No single candidate beats all others → e.g., A > B, B > C, C > A.
  - **Strategic manipulation(Gibbard–Satterthwaite theorem):** Voters can gain by misreporting preferences, leading to unfair outcomes.
  - **Short-term fairness:** Most methods ensure fairness only in one election; they ignore long-term representation in repeated elections.
- **Our project** aims to implement a system that provides:
  - **Fair winner selection**(Condorcet-compliant via Schulze method, with margin-based tie-breaking; if margins are equal, the lexicographically smallest candidate is chosen).
  - **Resistance to paradoxes**(resolving cycles using strongest paths).
  - Implement the **Perpetual Voting Algorithm** with pairwise comparisons.

Background

01

CONDORCET CRITERION:

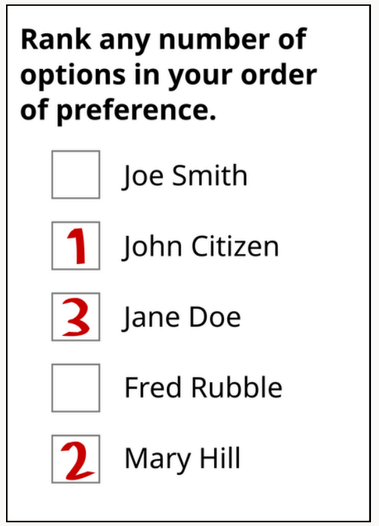
A candidate who wins every head-to-head comparison should be the overall winner.

02

TYPES OF BALLOTS:

- **Approval Ballots:**
  Voters select any number of candidates they approve of.
  Example: {A, C}
- **Ranked Ballots:**
  Voters rank candidates in order of preference.
  Example: A > C > B
- **Score Ballots:**
  Voters assign numerical scores to each candidate.
  Example: A= 5, C= 3, B= 0

Approval Ballot



Ranked Ballot

On a score ballot, the voter scores all the candidates.

Governor Candidates		Score each candidate by filling in a number (0 is worst; 9 is best)
1: Candidate A	→	⓪①②③④⑤⑥⑦⑧⑨
2: Candidate B	→	⓪①②③④⑤⑥⑦⑧⑨
3: Candidate C	→	⓪①②③④⑤⑥⑦⑧⑨

Score Ballot

03

SCHULZE METHOD:

- **Input:**
  System accepts Approval Ballots as input.
- **Builds a graph of pairwise victories.**
- **Uses path strengths to resolve cycles.**
- **Ensures that the strongest paths decide the winner.**

```
Enter candidates by giving space after every candidate: A B C D
Enter no of voters: 3
Enter no of rounds: 1


Round 1:
Voter 1 approval ballot (give space if more than one preference): A B
Voter 2 approval ballot (give space if more than one preference): B C D
Voter 3 approval ballot (give space if more than one preference): A C

Pairwise Preference Matrix (d)
0.00 1.50 1.50 2.00
1.50 0.00 1.50 1.50
1.50 1.50 0.00 1.50
1.00 0.50 0.50 0.00

Strongest Path Matrix (p)
0.00 0.00 0.00 2.00
0.00 0.00 0.00 1.50
0.00 0.00 0.00 1.50
0.00 0.00 0.00 0.00

Winner of Round 1: A

Winner Sequence:
{1: 'A'}
```

Pairwise Comparison Matrix and Strongest Path Matrix

```
candidates = ["A", "B", "C"]
voters = [
    {"A", "B"},
    {"B"},
    {"A", "C"},
]
```

Static Approval Ballot Input

Dynamic Approval Ballot Input

```
candidates_input = input("Enter candidates by giving space after every candidate: ")
candidates = candidates_input.split()
voters = int(input("Enter no of voters: "))
rounds = int(input("Enter no of rounds: "))
satisfactions = [0] * voters
for round_idx in range(rounds):
    weights = []
    for sat_v in satisfactions:
        weights.append(1 / (1 + sat_v))
    print(f"\n\nRound {round_idx + 1}:")
    approval_ballots = []
    for i in range(voters):
        approval=input(f"Voter {i+1} approval ballot (give space if more than one preference): ")
        approval_ballots.append(approval.split())
```

04

PERPETUAL EXTENSION:

- **Takes historical elections into account.**
- **Ensures long-term fairness and prevents dominance by a single group.**

Pre-Learning

Schulze Method from Wikipedia:

- Condorcet-compliant method that resolves cycles using strongest paths from research paper.
- Foundation for our perpetual extension.

AAAI 2020 Paper – Dynamic Ballots:

- Focuses on evolving preferences over time.
- Motivation for perpetual settings (voting across multiple rounds).

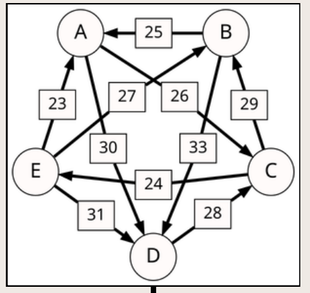
IJCAI 2018 Paper – Approval-based Multiwinner Rules:

- Introduced axioms like proportionality and long-term fairness.
- Helped us design approval-based perpetual variants.

Takeaway:

- Classical Condorcet methods handle single elections well,
- But real-world needs dynamic, perpetual fairness → motivates our work.

Schulze Method



Using Floyd Warshall

	$d[*, A]$	$d[*, B]$	$d[*, C]$	$d[*, D]$	$d[*, E]$
$d[A, *]$		20	26	30	22
$d[B, *]$	25		16	33	18
$d[C, *]$	19	29		17	24
$d[D, *]$	15	12	28		14
$d[E, *]$	23	27	21	31	

Strongest Path Matrix

Methodology

- **Input Collection** – Voters have preferences whether to approve or disapprove the candidates**(Approval Ballots)**.
- **Pairwise Comparison Matrix** – Construct a matrix showing how many voters prefer candidate A over candidate B.
- **Strongest Path Calculation** – Use **Floyd–Warshall algorithm** to find the strongest path between candidates.
- **Winner Determination** – Candidate who is unbeaten in strongest path comparisons becomes the winner.
- **Perpetual Extension** – Results are updated across multiple elections to ensure historical fairness.

Implementation Details

Language: **Python**

Core Algorithm:

- Updating **Satisfaction Weights** in each round.
- **Pairwise Matrix** Construction – **$O(n^2)$** comparisons.
- **Strongest Path** Calculation **(Floyd–Warshall)** – **$O(n^3)$**.
- Winner Determination – Schulze rules applied.
- Perpetual Extension – Maintains history across rounds.

Weighting ΔAB Scheme:

- **$\Delta = 1$** → If A is in approval ballot but not B.
- **$\Delta = 0.5$** → If A and B both are in approval ballot.
- **$\Delta = 0$** → None of them are in approval ballot.

Static vs Dynamic Handling:

- **Static** → Approval Ballots remain fixed for all the rounds.
- **Dynamic** → Approval Ballots changes every round (captures real-world variation).

Datasets Used

Poland Election Dataset (Static Ballots)

- Real-world dataset with approval preferences.
- Verified correctness of Schulze implementation.

Spotify Dataset (Static Ballots)

- Adapted user playlist preferences into approval sets.
- Used as test case for candidate popularity analysis.

User Input Simulation (Dynamic Ballots)

- Interactive input: voters could change preferences round-by-round.
- Demonstrated perpetual fairness across multiple cycles.

Synthetic Dataset Generation

- Inspired by real-world elections and recommender data.
- Goal: create benchmark data to test dynamic approval ballots.

Results & Observations

Static Ballots:

- If one candidate appears in every approval ballot → always wins.
- Verified using Spotify dataset.

Dynamic Ballots:

- Winners change with evolving voter approvals.
- Demonstrated robustness against dominance by single candidate.

Example Output (from code):

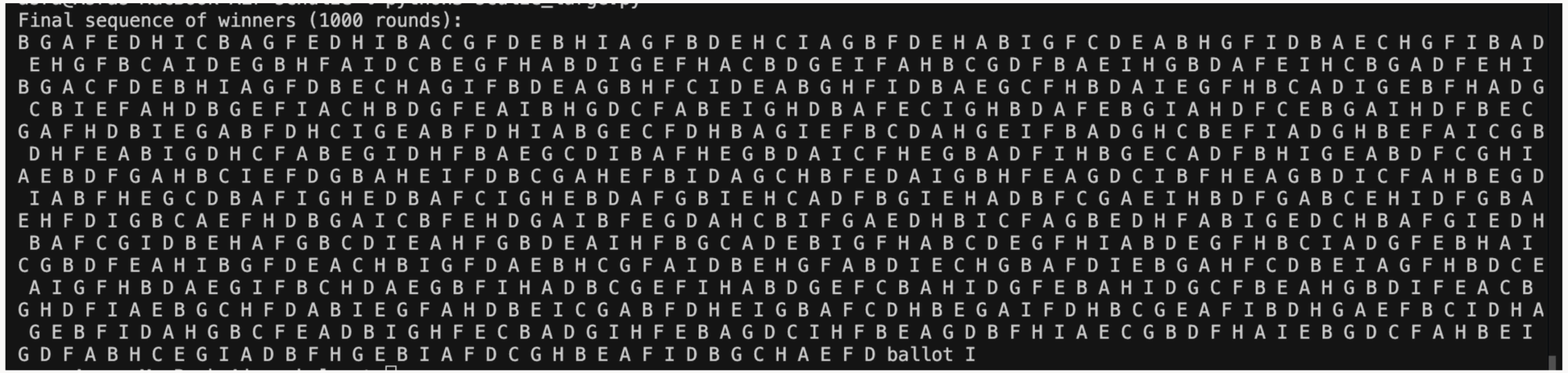
- Pairwise matrix generated correctly.
- Strongest path matrix resolves cycles fairly.
- In case of cyclic tie, the one with a win of largest margin is declared winner. If tie persists, lexicographically smallest candidate is the winner.

Comparative Analysis:

- Schulze + Perpetual method provides long-term fairness.
- Outperforms plurality/Borda in avoiding paradoxes.

Outputs

Output Of Synthetic Dataset Generation:



Output Of Poland Dataset:

Axiomatic Analysis

☛ **Axioms:** Basic fairness principles used to evaluate voting rules.

☛ **Following are the three axioms:**

- **Simple Proportionality:**
  Groups should be represented in proportion to their size.
  Example: If 40% voters support green, around 40% of seats should go to green.

- **Independence of Unanimous Decisions:**
  Unanimous agreement must be respected in the outcome.
  Example: If everyone approves candidate A, then A must be selected.

- **Bounded Dry Spells:**
  No large group should be excluded from representation for too long.
  Example: A group with 25% support should not keep losing every round indefinitely

Applications

- Widely applicable in online communities (Wikipedia, Debian elections).
- Multi-round organizational elections where fairness across time matters.
- Student bodies and councils in universities.
- Policy decision-making in academic and professional institutions.
- Any scenario needing fair, transparent, and manipulable-resistant elections.

Conclusion

- Successfully implemented Perpetual Schulze algorithm using pairwise comparisons for both static and dynamic approval ballots.
- Demonstrated ability to handle cycles and paradoxes effectively.
- Ensures fairness in single elections and long-term fairness in repeated elections.
- Results validate the method's robustness, correctness, and applicability.
- Provides a strong foundation for real-world adoption in democratic systems.

Plan Ahead

Explore Δ-weighting variations:

Investigate different ways of assigning weights to preferences to ensure greater fairness.

Long-Term Goals:

- Extend to multi-winner elections
- Compare performance with other perpetual voting rules.

References

- **Schulze Method:** Wikipedia
- **Perpetual Voting Method:**
  - Optimal Bounds for Dissatisfaction in Perpetual Voting
  - Perpetual Voting: The Axiomatic Lens
  - Justified Representation for Perpetual Voting
  - Artificial Delegates Resolve Fairness Issues with Partial Turnout
  - Perpetual Voting: Fairness in Long-Term Decision Making
  - Proportional Aggregation of Preferences for Sequential Decision Making
  - Proportional Decisions in Perpetual Voting
  - Temporal Fairness in Multiwinner Voting
- **Datasets:**
  - Poland Local Elections
  - Spotify Countries Chart
- **Images:**
  https://pngtree.com/freepng/voting-process-and-vote-election-flat-vector-illustration_14961531.html
  https://pngtree.com/freepng/word-cloud---voting-system-proportion-weight-word-cloud-vector_9504112.html
  https://en.wikipedia.org/wiki/Approval_ballot
  https://en.wikipedia.org/wiki/Ranked_voting
  https://en.wikipedia.org/wiki/Score_voting